Explicit Euler

First-order forward discretization

1 Numerical map

For an initial-value problem

$$\dot{z} = f(t, z), \qquad z(t_n) = z_n,$$

Taylor expansion gives

$$z(t_n + h) = z_n + hf(t_n, z_n) + \mathcal{O}(h^2).$$

Explicit Euler discards the remainder and defines

$$\Phi_{h,t_n}(z_n) = z_n + hf(t_n, z_n).$$

The method requires one vector-field evaluation, has local truncation error $\mathcal{O}(h^2)$ and, under the standard stability and smoothness assumptions, global error $\mathcal{O}(h)$.

2 Stability

For the test equation $y' = \lambda y$ the amplification factor is

$$R(\xi) = 1 + \xi, \qquad \xi = h\lambda.$$

Absolute stability requires $|1 + \xi| \leq 1$, the disk centered at -1 with radius one. The bounded stability region makes this method unsuitable for stiff problems unless the step is very small.

3 Hamiltonian interpretation

For a canonical Hamiltonian system with Jacobian $A = Df(z_n)$, the step tangent is $D\Phi = I + hA$. Even when A is Hamiltonian, $(D\Phi)^T \Omega D\Phi = \Omega + h^2 A^T \Omega A$ in general. Explicit Euler is therefore neither symmetric nor symplectic and does not preserve energy in general. It serves as a transparent first-order baseline for accuracy and geometric diagnostics.

4 Implemented configuration

One forward Euler stage. Energy tracking is optional and passive. Forward Euler evaluates the passive derivative at the start of the accepted step. The common state conventions are specified below.

5 Implementation correspondence

The fixed tableau is declared in `src/methods/runge_kutta/euler.py`. The fixed-grid execution lifecycle is documented in the sibling `simulation/` directory; shared physical dynamics are documented outside the model tree.

The recipe inherits stage calculations from `ExplicitRungeKutta`; `RungeKutta` supplies common step results and passive stage quadrature. The explicit algorithm consumes the immutable coefficients A, B and C without a separate implementation of this recipe.

6 Physical state and endpoint energy observations

Accepted states contain one physical vector $Z \in \mathbb{R}^{dN}$ and scalar time. GC vectors have $2N$ entries and FC vectors have $4N$; doubled workspaces are transient. InitialConditions supplies values and layout during preparation only. The numerical base retains functions, metadata, run status and copy count; execution owns the initial snapshot and output schedule. Supplied Jacobians are independent particle blocks (N, d, d); finite differences use dense matrices.

The generic `EnergyObserver` evaluates only

$$H_n = H(t_n, Z_n), \quad H_{n+1} = H(t_{n+1}, Z_{n+1}), \quad (\partial_t H)_n, \quad (\partial_t H)_{n+1}, \quad \Delta H_n = H_{n+1} - H_n.$$

It also evaluates these physical functions at requested saved samples, without consuming internal stages or integrating a momentum coordinate. The derivative is positive, analytic and partial at fixed state. GC differentiates its normalized Hamiltonian potential with `dt=1`; spatial splines and Fourier time factors retain their original scaling. This is not the secant $\Delta H_n/h$. Variation of a time-dependent Hamiltonian is not a numerical conservation error.

Specialized historical generalized-energy studies separately reconstruct $\dot{\kappa} = -\partial_t H$ from retained numerical stages. Their signed weights, physical normalization and archived diagnostics keep their original meaning. They are not part of generic energy observation. Host and JAX execution share the fixed numerical map and physical-only accepted state. JAX evaluates endpoint energy on the device without a momentum accumulator.

7 Optional JAX execution

An optional JAX driver executes the same numerical map on one selected CPU or GPU, with particle operations batched inside a compiled sequential time loop. It reuses the canonical coefficients and prepared potential splines. Physical-state tolerance scaling, signed stage times, passive energy weights and independent shortened output maps retain their definitions above. Applicable Newton systems use independent particle blocks; an explicitly selected Broyden iteration retains its global secant matrix. Nonlinear iteration limits and failure checks remain active on the device.

Both execution routes use double precision and return the same physical solution layout. Floating-point reassociation can produce round-off differences without changing the designed order or geometric properties. First-call compilation is separate from warmed integration timings, which include transfers and output construction. The sibling runtime document and shared execution contract specify supported callbacks and exact device scope.